ECSE 420 Parallel Computing, Assignment 1, Fall 2026

Student 1 name: Roman Humenyuk

Student 1 ID: 261240585

Student 2 name: Ziang Wang

Student 2 ID: 261161981

How to run each file and which question each one solves

All source files are in the package `ca.mcgill.ecse420.a1`. From the source root (the folder that contains `ca/`), compile with:

```
javac ca/mcgill/ecse420/a1/*.java
```

then run each part with the command shown below (from the same folder):

- `java ca.mcgill.ecse420.a1.MatrixMultiplication validate` → Q1.1, Q1.2 (correctness checks)
- `java ca.mcgill.ecse420.a1.MatrixMultiplication` → Q1.3 (one timed run, 2000×2000)
- `java ca.mcgill.ecse420.a1.MatrixMultiplication threads 4000` → Q1.4
- `java ca.mcgill.ecse420.a1.MatrixMultiplication sizes <t*>` → Q1.5 (t* = best thread count from Q1.4)
- `java ca.mcgill.ecse420.a1.Deadlock` → Q2.1
- `java ca.mcgill.ecse420.a1.Deadlock fixed` → Q2.2
- `java ca.mcgill.ecse420.a1.DiningPhilosophers deadlock 8` → Q3.1
- `java ca.mcgill.ecse420.a1.DiningPhilosophers nodeadlock 8` → Q3.2 (deadlock-free version)
- `java ca.mcgill.ecse420.a1.DiningPhilosophers nostarvation 8` → Q3.2 (starvation-free version)

For DiningPhilosophers the optional arguments are `[mode] [seconds] [philosophers]` (defaults: nostarvation, 8 s, 5 philosophers). No input files are needed.

Question 1: Matrix multiplication

1.1 `sequentialMultiplyMatrix` (Appendix A, `MatrixMultiplication.java`) implements the definition of the matrix product: for an n×m matrix a and an m×p matrix b, the result is the n×p matrix with result[i][j] = Σ a[i][k]·b[k][j] (sum over k = 0 … m − 1). The method uses three nested loops: the outer two iterate over every cell (i, j) of the result, and the inner loop computes the

dot product of row i of a with column j of b, accumulating into a local variable `sum` before storing it in result[i][j].

Validation. The method is checked by the `validate()` method, run with `java ca.mcgill.ecse420.a1.MatrixMultiplication validate`:

1. Hand-computed cases: a 2×2 product ([[1,2],[3,4]]·[[5,6],[7,8]] = [[19,22],[43,50]]) and a non-square 2×3 · 3×2 product ([[1,2,3],[4,5,6]]·[[7,8],[9,10],[11,12]] = [[58,64],[139,154]]), whose expected values were calculated by hand.
2. Algebraic identities on a random 50×50 matrix M: M·I = M and M·0 = 0.
3. Shape check: a 301×200 by 200×157 product has 301 rows and 157 columns.

All checks passed. The full console output (which also covers the parallel checks of 1.2) is:

```
PASS  2x2 hand-computed (sequential)
PASS  2x2 hand-computed (parallel, 2 threads)
PASS  2x3 * 3x2 hand-computed (sequential)
PASS  2x3 * 3x2 hand-computed (parallel, 2 threads)
PASS  M * I = M (sequential)
PASS  M * I = M (parallel, 4 threads)
PASS  M * 0 = 0 (sequential)
PASS  M * 0 = 0 (parallel, 4 threads)
PASS  301x200 * 200x157 gives a 301x157 result
PASS  parallel (1 threads) equals sequential
PASS  parallel (2 threads) equals sequential
PASS  parallel (3 threads) equals sequential
PASS  parallel (7 threads) equals sequential
PASS  parallel (16 threads) equals sequential
PASS  parallel (500 threads) equals sequential
ALL CHECKS PASSED
```

1.2 *Task definition.* `parallelMultiplyMatrix(a, b, threads)` splits the rows of the result matrix into `threads` contiguous blocks. With n rows and t threads, each block has ⌊n/t⌋ rows, and the first n mod t blocks receive one extra row, so every row is covered exactly once. The thread count is capped at the number of rows so that no thread is given an empty block. Each block becomes one `Callable` task that computes result[i][j] for every row i in its block, using the same triple loop as the sequential version. The tasks are run on a fixed-size thread pool (`Executors.newFixedThreadPool`). The two-argument `parallelMultiplyMatrix(a, b)` required by the assignment calls this method with `NUMBER_THREADS`.

Combining results. All tasks read the input matrices a and b (never modified) and write into one shared result array. Because the row ranges are disjoint, no two tasks ever write the same cell, so no locks are needed: the result is combined simply by each task filling in its own rows of the same array. The main thread submits the tasks with `invokeAll`, which blocks until every task has finished, and then calls `get()` on each returned `Future`. According to the `ExecutorService` memory-consistency guarantee, the actions of a task happen-before the return of `Future.get()`, so all writes made by the worker threads are visible to the main thread when the

result is returned. `get()` also rethrows any exception from a task, so a failed task cannot silently produce a half-filled matrix. The pool is shut down in a `finally` block.

Validation. Besides the hand-computed and identity cases from 1.1 (also run through the parallel method), the key test compares the parallel result with the sequential result on a random non-square 301×200 by 200×157 product, using 1, 2, 3, 7, 16 and 500 threads. The counts 3 and 7 test the uneven split (301 is not divisible by 3 or 7), and 500 > 301 tests the cap on the number of threads. Every parallel result was identical to the sequential result (see the output in 1.1), which confirms both the row partitioning and the absence of race conditions.

1.3 `measureExecutionTime(a, b)` records `System.nanoTime()` immediately before and after each of the two multiplications and prints the elapsed wall-clock time in milliseconds for the sequential and parallel methods. Output of `java ca.mcgill.ecse420.a1.MatrixMultiplication`:

```
Matrix size 2000x2000, [t*] thread(s)
  Sequential: [....] ms
  Parallel:   [....] ms
```

[Replace the three lines above with your own console output.]

For the experiments in 1.4 and 1.5, `averageTimeMs` builds on the same idea with two refinements: one untimed warm-up run, so that JIT compilation and class loading do not inflate the first measurement, followed by the average of `REPETITIONS` = 3 timed runs to smooth out noise from the operating system scheduler.

Plausibility tests. We checked that the timings are believable:

- Scaling: the algorithm performs n^3 multiply-adds, so doubling n should multiply the time by about 8. From the 1.5 table, sequential 500×500 took [x] ms and 1000×1000 took [y] ms, a ratio of [y/x] ≈ 8.
- Speed-up: with p threads on a machine with c cores, the parallel time should be roughly the sequential time divided by min(p, c). With [p] threads on 1000×1000 we measured a speed-up of [z]×.
- Repeatability: the three timed runs of each measurement were within [r] % of each other.
- Sanity: 100×100 matrices take only a few milliseconds, while 4000×4000 matrices take [time], consistent with n^3 growth.

1.4 Test machine: Apple M1 Pro (8 cores: 6 performance + 2 efficiency, no simultaneous multithreading, so 8 logical processors), 16 GB RAM, OpenJDK 17.0.16, macOS 26.5. Command:

`java ca.mcgill.ecse420.a1.MatrixMultiplication threads 4000`. Results (4000×4000, average of 3 runs after one warm-up run):

```
threads,time_ms,speedup
1,[...],1.00
2,[...],[...]
4,[...],[...]
8,[...],[...]
16,[...],[...]
32,[...],[...]
```

[Replace the table above with your output, and insert the graph: execution time (y-axis) vs. number of threads (x-axis).]

Discussion. [Draft — to be checked against the measured numbers.] Execution time drops sharply from 1 to 4 threads, with close to linear speed-up ([e.g. 3.8× at 4 threads]), because each thread works on an independent block of rows with no synchronization, so the work is almost perfectly parallel and 4 threads can all run on performance cores. From 4 to 8 threads the gain is smaller ([speed-up at 8]). The M1 Pro has only 6 fast performance cores; with 8 threads, two of them run on the much slower efficiency cores. Because every thread gets an equal block of rows, the threads on the efficiency cores finish last and the whole multiplication has to wait for them (load imbalance). With 16 and 32 threads there are more threads than cores, so they time-share the 8 cores; the time stays roughly flat ([or improves slightly / gets slightly worse]). Smaller blocks can balance the work better between fast and slow cores, but this is offset by the extra thread creation, context switching and scheduling overhead. Other limits on the speed-up are that matrix multiplication is memory-intensive, so all threads compete for the same memory bandwidth and shared cache, and Amdahl's law: the sequential parts (allocating the result, creating the pool, waiting for the tasks) bound the achievable speed-up. The minimum time was obtained with [t*] threads, which is the value used in 1.5 and set as `NUMBER_THREADS`.

1.5 Command: `java ca.mcgill.ecse420.a1.MatrixMultiplication sizes [t*]`. Results (average of 3 runs):

```
size,sequential_ms,parallel_ms,speedup
100,[...],[...],[...]
200,[...],[...],[...]
500,[...],[...],[...]
1000,[...],[...],[...]
2000,[...],[...],[...]
3000,[...],[...],[...]
4000,[...],[...],[...]
```

[Replace the table above with your output, and insert the graph: execution time (y-axis) vs. matrix size (x-axis), one curve for sequential and one for parallel.]

Discussion. Both curves grow roughly cubically, as expected from the n^3 operation count: every time the size doubles, both times increase by about 8×. For the smallest matrices (100×100, 200×200) the parallel version gives little or no benefit [and may even be slower — check your numbers], because the fixed cost of creating a thread pool and dispatching tasks is comparable to the few milliseconds of actual computation. As the size grows, this overhead becomes negligible and the speed-up approaches the value found in 1.4 ([speed-up]× with [t*] threads). The sequential curve may also show a larger-than-8× jump between [e.g. 1000 and 2000], because the matrices no longer fit in the CPU cache: the i-j-k loop order reads column j of b with a stride of one full row, so cache misses become significant once b exceeds the cache size. The parallel version suffers from the same effect, but hides part of it by keeping several cores busy at once.

Question 2: Deadlock

2.1 Deadlock is a situation in which a set of threads can never make progress, because each one is waiting for a resource that is held by another thread in the same set. Four conditions must all hold at the same time for deadlock to occur (the Coffman conditions):

1. *Mutual exclusion* – a resource can be held by only one thread at a time.
2. *Hold and wait* – a thread holds at least one resource while waiting to acquire another.
3. *No preemption* – a resource cannot be forcibly taken away from the thread holding it; it must be released voluntarily.
4. *Circular wait* – there is a cycle of threads T1 → T2 → … → Tn → T1 in which each thread waits for a resource held by the next.

Consequences: the deadlocked threads block forever, so the work they were responsible for is never completed. Any resources they hold (locks, memory, connections) stay locked, which can spread the blockage to other threads that later need those resources. Deadlock does not raise an exception; the program simply appears to hang, which makes it hard to detect and often forces a restart of the application.

Program. In `Deadlock.java` (Appendix A), two `ReentrantLock` objects, `lockA` and `lockB`, are shared by two threads. Thread 1 acquires `lockA` first and then requests `lockB`; Thread 2 acquires `lockB` first and then requests `lockA` (both through the method `acquireBothLocks`). A 100 ms sleep after the first acquisition makes it practically certain (though not strictly guaranteed) that both threads have taken their first lock before either requests its second one. All four conditions are then satisfied: a `ReentrantLock` can be held by one thread at a time (mutual exclusion); each thread holds one lock while waiting for the other (hold and wait); Java never takes a lock away

from its owner (no preemption); and Thread 1 waits for a lock held by Thread 2 while Thread 2 waits for a lock held by Thread 1 (circular wait).

So that the program can end on its own instead of hanging, the two threads are daemon threads and `main` waits for them for at most 2 s. If they have not finished by then, it asks the JVM's deadlock detector (`ThreadMXBean.findDeadlockedThreads()`) which threads are deadlocked and prints them. Output of `java ca.mcgill.ecse420.a1.Deadlock`:

```
Thread 2: acquired lock B
Thread 1: acquired lock A
Thread 2: waiting for lock A while holding lock B
Thread 1: waiting for lock B while holding lock A
DEADLOCK detected after 2000 ms:
  Thread 1 is blocked on a lock owned by Thread 2
  Thread 2 is blocked on a lock owned by Thread 1
"Both threads finished." was never printed.
```

Both threads acquire their first lock and announce that they are waiting for the second; after that no thread makes progress, the JVM confirms that each thread is blocked on the lock owned by the other, and "Both threads finished." is never printed.

2.2 Since deadlock requires all four conditions, it can be prevented by breaking any one of them:

- *Lock ordering (breaks circular wait)* – assign a global order to all locks and require every thread to acquire locks in that order. A cycle then becomes impossible, because a thread holding a "later" lock can never wait for an "earlier" one.
- *Try-lock with timeout or back-off (breaks hold and wait / no preemption)* – use `tryLock()`; if the second lock cannot be obtained, release the first lock, wait, and retry. This avoids permanent blocking, though it may cause livelock if not combined with randomized back-off.
- *Acquire all resources at once (breaks hold and wait)* – request every lock a task needs in a single atomic step, or use one coarser-grained lock that protects both resources. Simpler, but reduces concurrency.
- *Avoid nested locking altogether* – restructure the code so that a thread never holds one lock while requesting another, e.g. by using higher-level concurrent data structures or message passing instead of shared locks.
- *Deadlock detection and recovery* – let deadlock happen but detect it (e.g. with `ThreadMXBean.findDeadlockedThreads()`) and abort or restart one of the threads. Used in databases, but generally a last resort.

The first solution applies directly to our program, because the deadlock exists only since the two threads acquire the locks in opposite orders. Running `java ca.mcgill.ecse420.a1.Deadlock`

`fixed` makes Thread 2 call `acquireBothLocks("Thread 2", lockA, "A", lockB, "B")`, i.e. both threads acquire `lockA` before `lockB`. Whichever thread gets `lockA` first can also get `lockB`, finishes, and releases both; the other thread then proceeds. Circular wait can no longer occur:

```
Thread 1: acquired lock A
Thread 1: waiting for lock B while holding lock A
Thread 1: acquired lock B
Thread 2: acquired lock A
Thread 2: waiting for lock B while holding lock A
Thread 2: acquired lock B
Both threads finished.
```

The try-lock approach would also work here, but lock ordering is the simplest and cheapest fix, because it adds no extra code paths and no retry logic.

Question 3: Dining philosophers

3.1 *Program.* `DiningPhilosophers.java` (Appendix A) models each philosopher as a `Runnable` executed by its own thread in a fixed thread pool, and each chopstick as a `ReentrantLock` shared by the two neighbouring philosophers: philosopher i uses chopsticks i (left) and (i + 1) mod n (right). A philosopher repeatedly thinks for a random time, picks up two chopsticks (acquires both locks), eats for a random time, and puts them down (releases both locks in `finally` blocks). Because a lock can only be held by one thread at a time, two philosophers can never hold the same chopstick at the same time. The program runs for a fixed time, then stops all philosophers (`shutdownNow()`, which works because chopsticks are acquired with `lockInterruptibly()`) and prints how many meals each one ate.

Deadlock. In `deadlock` mode every philosopher picks up the left chopstick first, then the right one, and holds the left chopstick while waiting for the right one. Consider the interleaving in which all n philosophers finish thinking and each acquires their left chopstick before any of them acquires their right one. At that point all n chopsticks are held, and each philosopher i is blocked on chopstick (i + 1) mod n, which is held by philosopher (i + 1) mod n. This is a circular wait: no philosopher can proceed, and none will ever release what they hold, so the program is stuck forever.

While the simulation runs, `main` checks every 100 ms with `ThreadMXBean.findDeadlockedThreads()` whether the philosophers are deadlocked. Output of

`java ca.mcgill.ecse420.a1.DiningPhilosophers deadlock 8` (the last chopstick pick-ups before the deadlock, then the summary):

```
Mode: deadlock, 5 philosophers, 8 s, ordered chopsticks: false, fair locks: false
...
Philosopher 0 picked up chopstick 0
Philosopher 4 picked up chopstick 4
Philosopher 3 picked up chopstick 3
Philosopher 1 picked up chopstick 1
Philosopher 2 picked up chopstick 2

DEADLOCK detected: every philosopher holds one chopstick and waits for the next one.
Meals eaten:
  Philosopher 0: 48
  Philosopher 1: 56
  Philosopher 2: 56
  Philosopher 3: 54
  Philosopher 4: 47
```

Each philosopher picked up their left chopstick and then no one ate again: the deadlock happened after only about 50 meals per philosopher, even though the program was allowed to run for 8 s.

3.2 We use resource ordering: the chopsticks are numbered 0 to n − 1, and every philosopher picks up their lower-numbered chopstick first (`Math.min(left, right)`, then `Math.max(left, right)`). This changes the order only for the last philosopher, who now picks up chopstick 0 before chopstick n − 1. It removes the circular-wait condition: a waiting philosopher always holds a lower-numbered chopstick than the one they are waiting for, so along any chain of waiting philosophers the chopstick numbers strictly increase, and the chain can never loop back on itself. Without a cycle there is no deadlock. Output of `java ca.mcgill.ecse420.a1.DiningPhilosophers nodeadlock 8`:

```
Mode: nodeadlock, 5 philosophers, 8 s, ordered chopsticks: true, fair locks: false
...

No deadlock occurred during the 8 s run.
Meals eaten:
  Philosopher 0: 493
  Philosopher 1: 550
  Philosopher 2: 576
  Philosopher 3: 649
  Philosopher 4: 502
```

The deadlock detector never fired during the run, and every philosopher ate hundreds of times.

3.2 (continued): starvation. *Is starvation possible?* Yes. The solution above is deadlock-free but not starvation-free. Deadlock-freedom only guarantees that *some* philosopher makes progress, not that *every* philosopher does. The default `ReentrantLock` is non-fair: when a chopstick is released, a neighbour who has just finished eating can pick it up again ahead of a philosopher who is already waiting for it. This can repeat indefinitely, so a philosopher can starve.

Solution. In `nostarvation` mode each chopstick is a fair lock, `new ReentrantLock(true)`, which grants the lock to the thread that has waited the longest (this applies to `lock()` and `lockInterruptibly()`). The resource ordering from 3.2 is kept, so the program is still deadlock-free. Each chopstick is shared by only two philosophers, so a waiting philosopher is first in line and gets the chopstick as soon as their neighbour releases it. The neighbour always releases it eventually, because eating time is finite and there is no deadlock. Therefore every hungry philosopher eventually gets both chopsticks and eats: the solution is starvation-free.

Test. Each philosopher counts their meals, and the program prints all the counts at the end of the run. A starving philosopher would have a count that stops growing while the others keep increasing. Output of `java ca.mcgill.ecse420.a1.DiningPhilosophers nostarvation 8`:

```
Mode: nostarvation, 5 philosophers, 8 s, ordered chopsticks: true, fair locks: true
...

No deadlock occurred during the 8 s run.
Meals eaten:
  Philosopher 0: 496
  Philosopher 1: 546
  Philosopher 2: 588
  Philosopher 3: 663
  Philosopher 4: 507
```

In the 8-second run with 5 philosophers the program never stalled and every philosopher ate roughly 500–660 meals. Testing gives evidence, not proof: starvation-freedom is guaranteed by the fair-lock argument above.

Question 4: Amdahl's law

Throughout, the program takes unit time when executed sequentially ($T_1 = 1$). If a fraction s of that time is sequential and the remaining 1 − s is perfectly parallelizable, then on n processors $T_n = s + (1 - s)/n$ and Amdahl's law gives the speed-up $S_n = T_1/T_n = 1 / (s + (1 - s)/n)$.

4.1 Here s = 0.3, so the parallel part is 1 − 0.3 = 0.7.

```
T_n = 0.3 + 0.7/n
S_n = 1 / (0.3 + 0.7/n)
    = n / (0.3n + 0.7)            (multiply top and bottom by n)
    = 10n / (3n + 7)              (multiply top and bottom by 10)
```

Check: n = 1 gives $S_1 = 10/10 = 1$ (no speed-up on one processor), and n = 8 gives $S_8 = 80/31 \approx 2.58$.

Limit with an unbounded number of processors: as $n \to \infty$, $0.7/n \to 0$, so

```
lim S_n = 1 / 0.3 = 10/3 ≈ 3.33   (n → ∞)
```

The 30 % sequential part caps the speed-up at 3.33×, no matter how many processors are added.

4.2 Original program: s = 0.4, so

```
s_n = 1 / (0.4 + 0.6/n)
```

The improved version decreases the sequential time percentage by a factor of k, so the sequential fraction becomes 0.4/k. The program still takes unit time sequentially, so the parallel fraction becomes 1 − 0.4/k:

```
s'_n = 1 / (0.4/k + (1 − 0.4/k)/n)
```

Require s'_n = 2·s_n. Taking reciprocals:

```
0.4/k + (1 − 0.4/k)/n = (0.4 + 0.6/n) / 2 = 0.2 + 0.3/n
0.4n/k + 1 − 0.4/k    = 0.2n + 0.3          (multiply both sides by n)
(0.4/k)(n − 1)        = 0.2n − 0.7
k = 0.4(n − 1) / (0.2n − 0.7)
k = 4(n − 1) / (2n − 7)
```

Validity. k must be positive, so 2n − 7 > 0, i.e. n ≥ 4. For n ≤ 3 the speed-up cannot be doubled at all: even removing the sequential part completely (k → ∞) gives s'_n → n, which is less than 2·s_n (e.g. n = 3: $2 \cdot s_3 = 3.33 > 3$). As n → ∞, k → 4/2 = 2. Check with n = 4: k = 12; $s_4 = 1/(0.4 + 0.15) = 1.818$ and $s'_4 = 1/(0.4/12 + (1 − 0.4/12)/4) = 1/0.275 = 3.636 = 2 \cdot s_4$ ✓.

4.3 Let p be the fraction of the (unit) sequential execution time taken by the sequential part. On n processors:

```
Original:  T_n  = p + (1 − p)/n
Modified:  T'_n = p/3 + (1 − p/3)/n      (sequential percentage decreased 3-fold)
```

Require T'_n = T_n / 2:

```
p/3 + (1 − p/3)/n = p/2 + (1 − p)/(2n)
2pn + 6 − 2p      = 3pn + 3 − 3p          (multiply both sides by 6n)
3 + p             = pn
p(n − 1)          = 3
p = 3 / (n − 1)
```

Validity. p is a fraction, so p ≤ 1, which requires n ≥ 4. Check with n = 7: p = 0.5; $T_7 = 0.5 + 0.5/7 = 0.5714$ and $T'_7 = 0.5/3 + (1 − 0.5/3)/7 = 0.2857 = T_7/2$ ✓.

Appendix A – Source Code

[To be added at the end: the final versions of MatrixMultiplication.java, Deadlock.java and DiningPhilosophers.java.]

Appendix B – AI Use Disclosure

[Draft — review and edit so it describes exactly what you did. Add any other AI tools you used.]

AI tool(s) used: Claude (Anthropic).

Nature of use: reviewing our completed code and report against the assignment handout and the submission instructions; suggesting code changes (a `validate` mode for the correctness tests, using `invokeAll` and `Future.get()` to collect the parallel results, renaming variables to follow the Google Java Style Guide, automatic deadlock detection with `ThreadMXBean`, and separate modes for the deadlocking, deadlock-free and starvation-free versions of the dining philosophers); restructuring and rewording the report for Questions 1–3; instructions for running the timing experiments; and checking the algebra in Question 4.

Verification activities: we read all the AI-suggested code line by line to make sure we understand it. We compiled and ran every program on our own machine (Apple M1 Pro): the `validate` mode compares the parallel result with the sequential result and with hand-computed products, the deadlock programs were checked to deadlock (and the fixed versions to finish), and the meal counts were checked for starvation. All timing results in the report were measured on our machine, not produced by the AI. The Question 4 formulas were checked by substituting numbers (e.g. n = 4, k = 12 exactly doubles the speed-up in 4.2).

Reflection:

- *Useful contribution:* it found inconsistencies we had missed: the report described a `validate` mode and variable names that did not exist in our code, and the Q2.1 sample output did not match what the program prints. Also restructuring parts of the report text here.
- *Limitation / error:* before it had the assignment handout, it claimed that our answers to 4.2 and 4.3 were wrong, assuming that only the sequential part's time is reduced while the parallel work stays the same. The handout says the sequential time *percentage* is decreased, with the sequential run still taking unit time, and under that wording our original answers are correct. It also first pasted example outputs from its own 2-core test environment instead of our machine.
- *How it was resolved:* we gave it the exact wording of Question 4, re-derived both answers under that wording and checked them numerically, kept our original answers and added the validity conditions. We replaced all program outputs with outputs from our own runs.